

Drawer Pattern

Version: 1.0
Component: DetailDrawer
Last Updated: 2025-01-XX

Overview

The Drawer Pattern defines when and how to use right-side slide-out drawers for viewing and editing entity details in TPI portals.

When Drawer vs Route

Use Drawer When

- ☒ **Viewing detail of list item**
 - User clicks table row → Drawer opens
 - Context: User remains in list view
 - Use case: Quick detail review
- ☒ **Quick edit forms (< 5 fields)**
 - Simple updates (status, assignee, priority)
 - Context: Minimal disruption to workflow
 - Use case: Status updates, assignments
- ☒ **Contextual actions**
 - Filter options, sort settings
 - Context: Secondary to main content
 - Use case: Advanced filters, column settings
- ☒ **Secondary information**
 - Comments, notes, related items
 - Context: Supplementary to main view
 - Use case: Activity feed, audit trail

Use Full Page Route When

- ☒ **Primary workflow**
 - Creating new entities
 - Context: Full attention required
 - Use case: New incident form, new release
- ☒ **Complex forms (> 5 fields)**

- Multi-step wizards
- Context: Requires full screen
- Use case: Configuration, settings

☒ **Primary content**

- Dashboard, main table view
- Context: Main page content
- Use case: Dashboard, list pages

☒ **Settings pages**

- Portal configuration
- Context: Dedicated settings area
- Use case: User settings, portal settings

Decision Matrix

Scenario	Fields	Complexity	Use
View entity detail	N/A	Low	Drawer
Quick status update	1-2	Low	Drawer
Edit entity	3-5	Low	Drawer
Edit entity	6+	Medium	Full Page
Create entity	Any	Any	Full Page
Multi-step form	Any	High	Full Page
Settings	Any	Any	Full Page

Width Rules

Standard Widths

Narrow (400px)

- Use when: Simple views, minimal content
- Content: Status, metadata, quick actions
- Example: User profile preview, quick info

Standard (600px)

- Use when: Default for most detail views
- Content: Entity details, simple forms
- Example: Incident details, task details

Wide (800px)

- Use when: Complex forms, rich content

- Content: Multi-column layouts, detailed forms
- Example: Release details, configuration forms

Extra Wide (1000px) (use sparingly)

- Use when: Very complex content, data-heavy
- Content: Large tables, complex dashboards
- Example: Analytics view, detailed reports

Responsive Behavior

Desktop ($\geq 1280\text{px}$)

- Full width as specified
- Sidebar remains visible
- Content area adjusts

Tablet (768px-1279px)

- Drawer: 100% width (overlay)
- Sidebar: Collapsed or hidden
- Backdrop: Full opacity

Mobile ($<768\text{px}$)

- Drawer: 100% width (full screen)
- Backdrop: Full opacity
- Close: Swipe down or X button

Close Behavior

Close Methods

Standard Close

- X button: Top-right corner
- Backdrop click: Closes drawer (if non-destructive)
- Escape key: Closes drawer
- Back button: Browser back (if deep-linked)

Prevent Close

- Unsaved changes: Show confirmation dialog
- Critical action in progress: Disable close
- Loading state: Disable close until complete

Unsaved Changes Handling

Detection

- Track: Form state vs. original data

- Indicator: "Unsaved changes" badge or asterisk
- Warning: Show on close attempt

Confirmation Dialog

- Title: "Unsaved changes"
- Message: "You have unsaved changes. Are you sure you want to close?"
- Actions: "Discard changes" (destructive) + "Cancel" (keep open)
- Default: "Cancel" (safe default)

Save on Close (optional)

- Auto-save: Save changes automatically
 - Use when: Non-critical edits
 - Feedback: Toast notification on save
-

Deep-Linking Approach

URL Structure

Standard Drawer

- No URL change: Drawer state in component state
- Use when: Quick views, temporary context

Deep-Linked Drawer

- URL: `/portal/section?drawer=[entityId]`
- Use when: Shareable links, bookmarkable views
- Example: `/cto/incidents?drawer=INC-2025-001`

Deep-Link Implementation

Opening Drawer

1. User clicks row → Update URL with `?drawer=[id]`
2. Drawer opens
3. Browser history: Entry added

Closing Drawer

1. User closes drawer → Remove `?drawer` from URL
2. Browser history: Entry added (back button works)

Direct Navigation

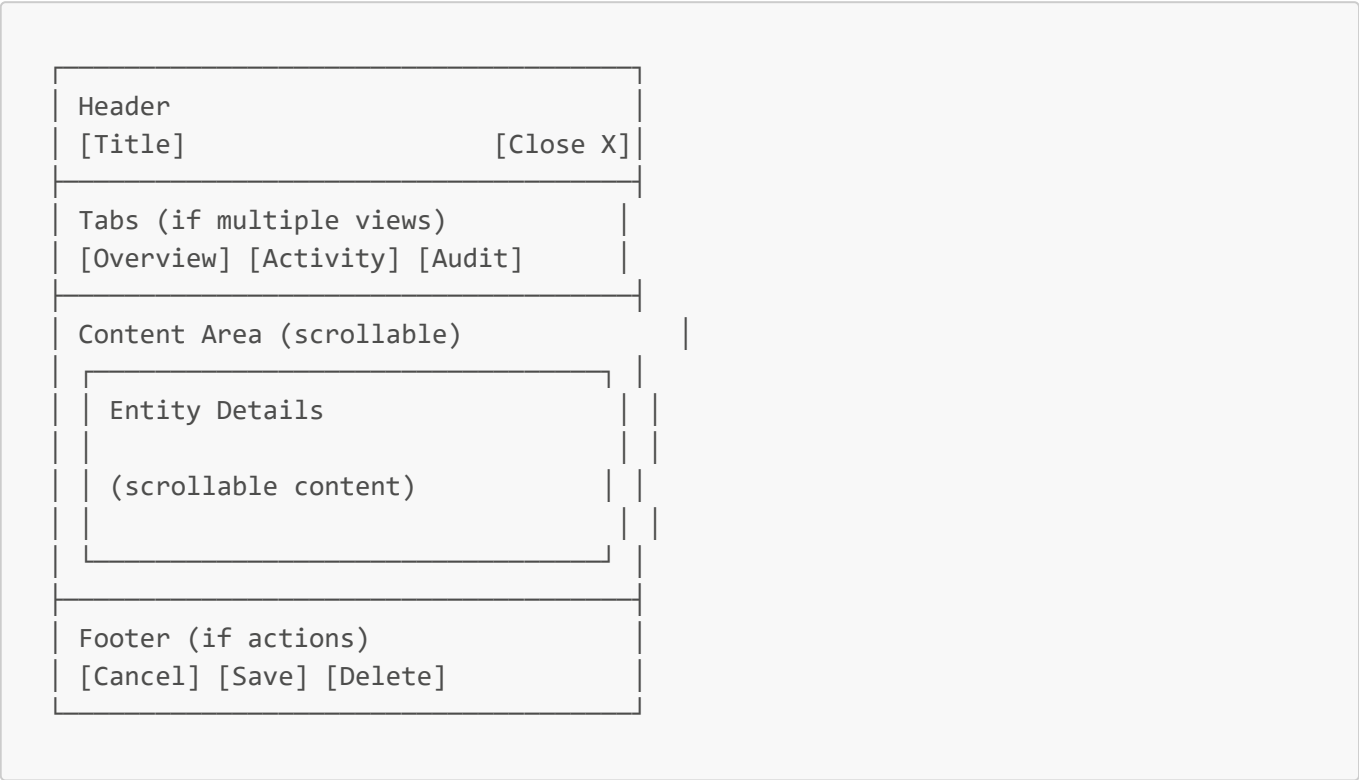
1. User visits URL with `?drawer=[id]`
2. Drawer opens automatically
3. Data loads for entity ID

Browser Back

- 1. User clicks back → URL changes
- 2. Drawer closes (if `?drawer` removed)
- 3. Drawer opens (if `?drawer` added)

Drawer Content Structure

Standard Layout



Header

Content

- Title: Entity name or identifier
- Subtitle: Optional, entity type or status
- Actions: Optional, quick actions (Edit, Share)

Styling

- Height: 64px
- Border: Bottom border (1px, subtle)
- Padding: 16px horizontal, 20px vertical

Tabs (Optional)

When to Use

- Multiple related views (Overview, Activity, Audit)
- Related entities (Comments, Attachments)
- Different edit modes (Basic, Advanced)

Tab Structure

- Default: First tab active
- Persist: Active tab in URL or state
- Navigation: Click tab → Content updates

Content Area

Scrolling

- Scrollable: If content exceeds drawer height
- Scroll indicator: Show when scrollable
- Scroll position: Maintain on tab switch

Spacing

- Padding: 24px (standard), 16px (compact)
- Section gaps: 24px between sections
- Form spacing: 16px between form fields

Footer

When to Show

- Editable content: Show Save/Cancel
- Actions available: Show action buttons
- Read-only: Hide footer

Button Layout

- Left: Cancel/Close button
- Right: Primary action (Save, Delete)
- Spacing: 12px between buttons

Animation & Transitions

Open Animation

Slide-in

- Direction: Right to left
- Duration: 300ms
- Easing: `ease-out`
- Backdrop: Fade in simultaneously

Implementation

```
@keyframes slideIn {  
  from {  
    transform: translateX(100%);
```

```
    }  
    to {  
      transform: translateX(0);  
    }  
  }  
}
```

Close Animation

Slide-out

- Direction: Left to right
- Duration: 250ms
- Easing: `ease-in`
- Backdrop: Fade out simultaneously

Backdrop

Opacity

- Default: 50% opacity
- Color: Black (#000000)
- Animation: Fade in/out with drawer

Interaction

- Clickable: Yes (closes drawer if non-destructive)
 - Non-clickable: When critical action in progress
-

Focus Management

Opening Drawer

Initial Focus

- First focusable element in drawer
- Usually: Close button or first input
- Exception: If form, focus first input

Focus Trap

- Tab: Cycles through drawer elements only
- Shift+Tab: Reverse cycle
- Escape: Closes drawer and returns focus to trigger

Closing Drawer

Return Focus

- Return to: Element that opened drawer
- Usually: Table row or button

- Fallback: Page header if trigger not available

Focus Indicator

- Visible: Always show focus indicator
 - Style: 2px solid primary color outline
-

Accessibility

ARIA Attributes

Drawer Container

- `role="dialog"`
- `aria-modal="true"`
- `aria-labelledby="drawer-title"`
- `aria-describedby="drawer-description"` (if applicable)

Close Button

- `aria-label="Close drawer"`
- Keyboard: Escape key

Backdrop

- `aria-hidden="true"` (when drawer open)
- Prevents screen reader from reading background content

Keyboard Navigation

Standard Navigation

- Tab: Next focusable element
- Shift+Tab: Previous focusable element
- Escape: Close drawer
- Enter: Activate button/link

Form Navigation

- Tab: Next form field
- Shift+Tab: Previous form field
- Enter: Submit form (if in form context)

Screen Reader Support

Announcements

- Opening: "Drawer opened: [title]"
- Closing: "Drawer closed"
- Content changes: Announce significant updates

Content Reading

- Read: Drawer title and description on open
 - Skip: Background content (aria-hidden)
 - Focus: Announce focused element
-

Performance Considerations

Lazy Loading

Content Loading

- Load: On drawer open (not on page load)
- Progressive: Load critical data first
- Lazy: Load secondary data after render

Component Loading

- Code splitting: Lazy load drawer content components
- Example: `React.lazy(() => import('./IncidentDetails'))`

Rendering Optimization

Virtual Scrolling

- Use when: Long lists in drawer (>50 items)
- Implementation: Virtual scroll component
- Performance: Render visible items only

Memoization

- Memoize: Expensive computations
 - Use: `React.memo` for drawer content components
 - Avoid: Unnecessary re-renders
-

Best Practices

Do

- ☒ Use drawer for quick detail views
- ☒ Provide clear close mechanism
- ☒ Handle unsaved changes gracefully
- ☒ Return focus to trigger on close
- ☒ Use appropriate width for content
- ☒ Provide keyboard navigation
- ☒ Announce drawer state to screen readers
- ☒ Lazy load drawer content

Don't

- ✗ Use drawer for primary workflows
 - ✗ Use drawer for complex multi-step forms
 - ✗ Hide important actions in drawer
 - ✗ Make drawer too wide (>1000px)
 - ✗ Block close without good reason
 - ✗ Forget to return focus on close
 - ✗ Load all drawer data on page load
 - ✗ Use drawer for settings pages
-

Implementation Example

```
const [selectedIncident, setSelectedIncident] = useState<string | null>(null);
const [unsavedChanges, setUnsavedChanges] = useState(false);

const handleRowClick = (incidentId: string) => {
  // Update URL for deep-linking
  navigate(`/cto/incidents?drawer=${incidentId}`);
  setSelectedIncident(incidentId);
};

const handleClose = () => {
  if (unsavedChanges) {
    // Show confirmation dialog
    setShowCloseConfirm(true);
  } else {
    // Close drawer and update URL
    navigate('/cto/incidents');
    setSelectedIncident(null);
  }
};

return (
  <>
    <DataTable
      data={incidents}
      onClick={handleRowClick}
    />

    <DetailDrawer
      open={!selectedIncident}
      onClose={handleClose}
      title={`Incident ${selectedIncident}`}
      entityId={selectedIncident}
      width={700}
      unsavedChanges={unsavedChanges}
      footer={
        <div className="flex justify-end gap-2">
          <Button variant="outline" onClick={handleClose}>
            Cancel
          </Button>
        </div>
      }
    />
  </>
)
```

```

        <Button onClick={handleSave}>
          Save Changes
        </Button>
      </div>
    }
  >
  <Tabs defaultValue="overview">
    <TabList>
      <TabTrigger value="overview">Overview</TabTrigger>
      <TabTrigger value="activity">Activity</TabTrigger>
      <TabTrigger value="audit">Audit Trail</TabTrigger>
    </TabList>
    <TabContent value="overview">
      <IncidentOverview
        id={selectedIncident}
        onDataChange={() => setUnsavedChanges(true)}
      />
    </TabContent>
    <TabContent value="activity">
      <ActivityFeed entityId={selectedIncident} />
    </TabContent>
    <TabContent value="audit">
      <AuditTrail entityId={selectedIncident} entityType="incident" />
    </TabContent>
  </Tabs>
</Drawer>

<ConfirmDialog
  open={showCloseConfirm}
  onClose={() => setShowCloseConfirm(false)}
  onConfirm={() => {
    setUnsavedChanges(false);
    handleClose();
  }}
  title="Unsaved Changes"
  message="You have unsaved changes. Are you sure you want to close?"
  variant="default"
/>
</>
);

```

Related:

- [Enterprise Table Pattern](#)
- [Component Inventory](#)